AFRILANG Stdlib

std/retry

Français. Bibliothèque AFRILANG ‘std/retry‘ (niveau simple, catégorie text). Elle expose 2 fonction(s) : backoffMs, shouldRetry.
‘import ”std/retry”‘ · ‘use retry‘
- ‘backoffMs(attempt number) → number‘ - ‘shouldRetry(attempt number, max number) → bool‘ **English.** AFRILANG library ‘std/retry‘ (tier simple, category text). Exports 2 function(s): backoffMs, shouldRetry.
‘import ”std/retry”‘ · ‘use retry‘
- ‘backoffMs(attempt number) → number‘ - ‘shouldRetry(attempt number, max number) → bool‘

Catégorie / Category	Texte & chaînes	
Niveau / Tier	simple	June 16, 2026
Fonctions / Functions	2	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/retry‘ (niveau simple, catégorie text). Elle expose 2 fonction(s) : backoffMs, shouldRetry.
‘import ”std/retry”‘ · ‘use retry‘

```
- `backoffMs(attempt number) → number`
- `shouldRetry(attempt number, max number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/retry"
use retry
```

```
Niveau : simple · Catégorie : Texte & chaînes
Manipulation de texte, regex, formatage, parsing.
```

3. Référence API

```
•
 backoffMs(attempt number) → number•
 shouldRetry(attempt number, max number) → bool
```

4. Exemples d’utilisation

```
import "std/retry"
use retry
```

```
create result = backoffMs(/* args */)
say result
```

```
create result = shouldRetry(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/retry"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module retry

```
function backoffMs(attempt number) returns number
end
```

```
function shouldRetry(attempt number, max number) returns bool
end
end
```

English

1. Overview

AFRILANG library ‘std/retry‘ (tier simple, category text). Exports 2 function(s): backoffMs, shouldRetry.

‘import ”std/retry”‘ · ‘use retry‘

```
- `backoffMs(attempt number) → number`
- `shouldRetry(attempt number, max number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/retry"
use retry
```

```
Tier: simple · Category: Text & strings
Text manipulation, regex, formatting, parsing.
```

3. API reference

```
•
 backoffMs(attempt number) → number•
 shouldRetry(attempt number, max number) → bool
```

4. Usage examples

```
import "std/retry"
use retry
```

```
create result = backoffMs(/* args */)
say result
```

```
create result = shouldRetry(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/retry"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module retry

```
function backoffMs(attempt number) returns number
end
```

```
function shouldRetry(attempt number, max number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/retry"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/retry/

Source (extrait)

```
module retry

function backoffMs(attempt number) returns number
end

function shouldRetry(attempt number, max number) returns bool
end
end
```